Image processing on FPGA using Verilog HDL

This FPGA project is aimed to show in details how to process an image using Verilog from reading an input bitmap image (.bmp) in Verilog, processing and writing the processed result to an output bitmap image in Verilog. **The full Verilog code for reading image, image processing, and writing image is provided.**

In this FPGA Verilog project, some simple processing operations are implemented in Verilog such as inversion, brightness control and threshold operations. The image processing operation is selected by a "parameter.v" file and then, the processed image data are written to a bitmap image output.bmp for verification purposes. The image reading Verilog code operates as a Verilog model of an image sensor/ camera, which can be really helpful for functional verifications in real-time FPGA image processing projects. The image writing part is also extremely useful for testing as well when you want to see the output image in BMP format. In this project, I added some simple image processing code into the reading part to make an example of image processing, but you can easily remove it to get raw image data. All the related questions asked by students are answered at the bottom of this article.

First of all, Verilog cannot read images directly. To read the .bmp image on in Verilog, the image is required to be converted from the bitmap format to the hexadecimal format. Below is a Matlab example code to convert a bitmap image to a .hex file. The input image size is 768x512 and the image .hex file includes R, G, B data of the bitmap image.

```
b=imread('kodim24.bmp'); % 24-bit BMP image RGB888

k=1;
for i=512:-1:1 % image is written from the last row to the first row
for j=1:768
a(k)=b(i,j,1);
a(k+1)=b(i,j,2);
a(k+2)=b(i,j,3);
k=k+3;
end
end
fid = fopen('kodim24.hex', 'wt');
fprintf(fid, '%x\n', a);
disp('Text file write done');disp(' ');
fclose(fid);
% fpga4student.com FPGA projects, Verilog projects, VHDL projects
```

To read the image hexadecimal data file, Verilog uses this command: $readmemh or $readmemb if the image data is in a binary text file. After reading the image .hex file, the RGB image data are saved into memory and read out for processing.

Below is the Verilog code to the image reading and processing part:

```
/*****************************************************************************/
/*****************  Module for reading and processing image     **************/
/*****************************************************************************/
`include "parameter.v"       // Include definition file
// fpga4student.com: FPGA projects for students
// FPGA project: Image processing in Verilog
module image_read
```

```
#(
  parameter     WIDTH  = 768,   // Image width
         HEIGHT  = 512,   // Image height
  INFILE  = "./img/kodim01.hex",  // image file
  START_UP_DELAY = 100, // Delay during start up time
  HSYNC_DELAY = 160,// Delay between HSYNC pulses
  VALUE= 100, // value for Brightness operation
  THRESHOLD= 90, // Threshold value for Threshold operation
  SIGN=1         // Sign value using for brightness operation
  // SIGN = 0: Brightness subtraction
  // SIGN = 1: Brightness addition
)
(
  input HCLK,          // clock
  input HRESETn,         // Reset (active low)
  output VSYNC,        // Vertical synchronous pulse
 // This signal is often a way to indicate that one entire image is transmitted.
 // Just create and is not used, will be used once a video or many images are tran
  output reg HSYNC,        // Horizontal synchronous pulse
 // An HSYNC indicates that one line of the image is transmitted.
 // Used to be a horizontal synchronous signals for writing bmp file.
  output reg [7:0]  DATA_R0,  // 8 bit Red data (even)
  output reg [7:0]  DATA_G0,  // 8 bit Green data (even)
  output reg [7:0]  DATA_B0,  // 8 bit Blue data (even)
  output reg [7:0]  DATA_R1,  // 8 bit Red  data (odd)
  output reg [7:0]  DATA_G1,  // 8 bit Green data (odd)
  output reg [7:0]  DATA_B1,  // 8 bit Blue data (odd)
  // Process and transmit 2 pixels in parallel to make the process faster, you can
  output     ctrl_done     // Done flag
);
//-----------------------------------------------
// Internal Signals
//-----------------------------------------------
parameter sizeOfWidth = 8;   // data width
parameter sizeOfLengthReal = 1179648;   // image data : 1179648 bytes: 512 * 768 *
// local parameters for FSM
localparam  ST_IDLE  = 2'b00,// idle state
   ST_VSYNC = 2'b01,// state for creating vsync
   ST_HSYNC = 2'b10,// state for creating hsync
   ST_DATA  = 2'b11;// state for data processing
reg [1:0] cstate,   // current state
nstate;    // next state
reg start;         // start signal: trigger Finite state machine beginning to oper
reg HRESETn_d;   // delayed reset signal: use to create start signal
reg   ctrl_vsync_run; // control signal for vsync counter
reg [8:0] ctrl_vsync_cnt; // counter for vsync
reg   ctrl_hsync_run; // control signal for hsync counter
reg [8:0] ctrl_hsync_cnt; // counter  for hsync
reg   ctrl_data_run; // control signal for data processing
reg [7 : 0]   total_memory [0 : sizeOfLengthReal-1];// memory to store  8-bit data
// temporary memory to save image data : size will be WIDTH*HEIGHT*3
integer temp_BMP   [0 : WIDTH*HEIGHT*3 - 1];
integer org_R  [0 : WIDTH*HEIGHT - 1];  // temporary storage for R component
integer org_G  [0 : WIDTH*HEIGHT - 1]; // temporary storage for G component
integer org_B  [0 : WIDTH*HEIGHT - 1]; // temporary storage for B component
// counting variables
integer i, j;
// temporary signals for calculation: details in the paper.
integer tempR0,tempR1,tempG0,tempG1,tempB0,tempB1; // temporary variables in contr

integer value,value1,value2,value4;// temporary variables in invert and threshold
reg [ 9:0] row; // row index of the image
reg [10:0] col; // column index of the image
```

```
reg [18:0] data_count; // data counting for entire pixels of the image
//-----------------------------------------------//
// -------- Reading data from input file ----------//
//-----------------------------------------------//
initial begin
    $readmemh(INFILE,total_memory,0,sizeOfLengthReal-1); // read file from INFILE
end
// use 3 intermediate signals RGB to save image data
always@(start) begin
    if(start == 1'b1) begin
        for(i=0; i<WIDTH*HEIGHT*3 ; i=i+1) begin
            temp_BMP[i] = total_memory[i+0][7:0];
        end

        for(i=0; i<HEIGHT; i=i+1) begin
            for(j=0; j<WIDTH; j=j+1) begin
     // Matlab code writes image from the last row to the first row
     // Verilog code does the same in reading to correctly save image pixels into
                org_R[WIDTH*i+j] = temp_BMP[WIDTH*3*(HEIGHT-i-1)+3*j+0]; // save R
                org_G[WIDTH*i+j] = temp_BMP[WIDTH*3*(HEIGHT-i-1)+3*j+1];// save Gr
                org_B[WIDTH*i+j] = temp_BMP[WIDTH*3*(HEIGHT-i-1)+3*j+2];// save Bl
            end
        end
    end
end
//---------------------------------------------------//
// ---Begin to read image file once reset was high ---//
// ---by creating a starting pulse (start)------------//
//---------------------------------------------------//
always@(posedge HCLK, negedge HRESETn)
begin
    if(!HRESETn) begin
        start <= 0;
  HRESETn_d <= 0;
    end
    else begin           //          ______
        HRESETn_d <= HRESETn;       //        |  |
  if(HRESETn == 1'b1 && HRESETn_d == 1'b0)  // __0___| 1 |___0____ : starting puls
   start <= 1'b1;
  else
   start <= 1'b0;
    end
end
//-------------------------------------------------------------------------------
// Finite state machine for reading RGB888 data from memory and creating hsync and
//-------------------------------------------------------------------------------
always@(posedge HCLK, negedge HRESETn)
begin
    if(~HRESETn) begin
        cstate <= ST_IDLE;
    end
    else begin
        cstate <= nstate; // update next state
    end
end
//----------------------------------------//
//--------- State Transition --------------//
//----------------------------------------//
// IDLE . VSYNC . HSYNC . DATA
always @(*) begin
 case(cstate)
  ST_IDLE: begin
   if(start)
```

```
    nstate = ST_VSYNC;
   else
    nstate = ST_IDLE;
  end
  ST_VSYNC: begin
   if(ctrl_vsync_cnt == START_UP_DELAY)
    nstate = ST_HSYNC;
   else
    nstate = ST_VSYNC;
  end
  ST_HSYNC: begin
   if(ctrl_hsync_cnt == HSYNC_DELAY)
    nstate = ST_DATA;
   else
    nstate = ST_HSYNC;
  end
  ST_DATA: begin
   if(ctrl_done)
    nstate = ST_IDLE;
   else begin
    if(col == WIDTH - 2)
     nstate = ST_HSYNC;
    else
     nstate = ST_DATA;
   end
  end
 endcase
end
// ------------------------------------------------------------------ //
// --- counting for time period of vsync, hsync, data processing ----  //
// ------------------------------------------------------------------ //
always @(*) begin
 ctrl_vsync_run = 0;
 ctrl_hsync_run = 0;
 ctrl_data_run  = 0;
 case(cstate)
  ST_VSYNC:  begin ctrl_vsync_run = 1; end  // trigger counting for vsync
  ST_HSYNC:  begin ctrl_hsync_run = 1; end // trigger counting for hsync
  ST_DATA:  begin ctrl_data_run  = 1; end // trigger counting for data processing
 endcase
end
// counters for vsync, hsync
always@(posedge HCLK, negedge HRESETn)
begin
    if(~HRESETn) begin
        ctrl_vsync_cnt <= 0;
  ctrl_hsync_cnt <= 0;
    end
    else begin
        if(ctrl_vsync_run)
   ctrl_vsync_cnt <= ctrl_vsync_cnt + 1; // counting for vsync
  else
   ctrl_vsync_cnt <= 0;

        if(ctrl_hsync_run)
   ctrl_hsync_cnt <= ctrl_hsync_cnt + 1; // counting for hsync
  else
   ctrl_hsync_cnt <= 0;
    end
end
// counting column and row index  for reading memory
always@(posedge HCLK, negedge HRESETn)
begin
```

```
    if(~HRESETn) begin
        row <= 0;
  col <= 0;
    end
 else begin
  if(ctrl_data_run) begin
   if(col == WIDTH - 2) begin
    row <= row + 1;
   end
   if(col == WIDTH - 2)
    col <= 0;
   else
    col <= col + 2; // reading 2 pixels in parallel
  end
 end
end
//-------------------------------------------------//
//----------------Data counting---------- ---------//
//-------------------------------------------------//
always@(posedge HCLK, negedge HRESETn)
begin
    if(~HRESETn) begin
        data_count <= 0;
    end
    else begin
        if(ctrl_data_run)
   data_count <= data_count + 1;
    end
end
assign VSYNC = ctrl_vsync_run;
assign ctrl_done = (data_count == 196607)? 1'b1: 1'b0; // done flag
//-------------------------------------------------//
//-------------  Image processing   ---------------//
//-------------------------------------------------//
always @(*) begin

 HSYNC   = 1'b0;
 DATA_R0 = 0;
 DATA_G0 = 0;
 DATA_B0 = 0;
 DATA_R1 = 0;
 DATA_G1 = 0;
 DATA_B1 = 0;
 if(ctrl_data_run) begin

  HSYNC   = 1'b1;
  `ifdef BRIGHTNESS_OPERATION
  /*************************************/
  /*  BRIGHTNESS ADDITION OPERATION */
  /*************************************/
  if(SIGN == 1) begin
  // R0
  tempR0 = org_R[WIDTH * row + col   ] + VALUE;
  if (tempR0 > 255)
   DATA_R0 = 255;
  else
   DATA_R0 = org_R[WIDTH * row + col   ] + VALUE;
  // R1
  tempR1 = org_R[WIDTH * row + col+1   ] + VALUE;
  if (tempR1 > 255)
   DATA_R1 = 255;
  else
   DATA_R1 = org_R[WIDTH * row + col+1   ] + VALUE;
```

```
 // G0
 tempG0 = org_G[WIDTH * row + col   ] + VALUE;
 if (tempG0 > 255)
  DATA_G0 = 255;
 else
  DATA_G0 = org_G[WIDTH * row + col   ] + VALUE;
 tempG1 = org_G[WIDTH * row + col+1   ] + VALUE;
 if (tempG1 > 255)
  DATA_G1 = 255;
 else
  DATA_G1 = org_G[WIDTH * row + col+1   ] + VALUE;
 // B
 tempB0 = org_B[WIDTH * row + col   ] + VALUE;
 if (tempB0 > 255)
  DATA_B0 = 255;
 else
  DATA_B0 = org_B[WIDTH * row + col   ] + VALUE;
 tempB1 = org_B[WIDTH * row + col+1   ] + VALUE;
 if (tempB1 > 255)
  DATA_B1 = 255;
 else
  DATA_B1 = org_B[WIDTH * row + col+1   ] + VALUE;
end
else begin
/*************************************/
/* BRIGHTNESS SUBTRACTION OPERATION */
/*************************************/
 // R0
 tempR0 = org_R[WIDTH * row + col   ] - VALUE;
 if (tempR0 < 0)
  DATA_R0 = 0;
 else
  DATA_R0 = org_R[WIDTH * row + col   ] - VALUE;
 // R1
 tempR1 = org_R[WIDTH * row + col+1   ] - VALUE;
 if (tempR1 < 0)
  DATA_R1 = 0;
 else
  DATA_R1 = org_R[WIDTH * row + col+1   ] - VALUE;
 // G0
 tempG0 = org_G[WIDTH * row + col   ] - VALUE;
 if (tempG0 < 0)
  DATA_G0 = 0;
 else
  DATA_G0 = org_G[WIDTH * row + col   ] - VALUE;
 tempG1 = org_G[WIDTH * row + col+1   ] - VALUE;
 if (tempG1 < 0)
  DATA_G1 = 0;
 else
  DATA_G1 = org_G[WIDTH * row + col+1   ] - VALUE;
 // B
 tempB0 = org_B[WIDTH * row + col   ] - VALUE;
 if (tempB0 < 0)
  DATA_B0 = 0;
 else
  DATA_B0 = org_B[WIDTH * row + col   ] - VALUE;
 tempB1 = org_B[WIDTH * row + col+1   ] - VALUE;
 if (tempB1 < 0)
  DATA_B1 = 0;
 else
  DATA_B1 = org_B[WIDTH * row + col+1   ] - VALUE;
 end
 `endif
```

```
        /*************************************/
  /*  INVERT_OPERATION       */
  /*************************************/
  `ifdef INVERT_OPERATION
   value2 = (org_B[WIDTH * row + col  ] + org_R[WIDTH * row + col  ] +org_G[WIDTH
   DATA_R0=255-value2;
   DATA_G0=255-value2;
   DATA_B0=255-value2;
   value4 = (org_B[WIDTH * row + col+1  ] + org_R[WIDTH * row + col+1  ] +org_G[WI
   DATA_R1=255-value4;
   DATA_G1=255-value4;
   DATA_B1=255-value4;
  `endif
  /*************************************/
  /********THRESHOLD OPERATION  *********/
  /*************************************/
  `ifdef THRESHOLD_OPERATION
  value = (org_R[WIDTH * row + col   ]+org_G[WIDTH * row + col   ]+org_B[WIDTH * r
  if(value > THRESHOLD) begin
   DATA_R0=255;
   DATA_G0=255;
   DATA_B0=255;
  end
  else begin
   DATA_R0=0;
   DATA_G0=0;
   DATA_B0=0;
  end
  value1 = (org_R[WIDTH * row + col+1   ]+org_G[WIDTH * row + col+1   ]+org_B[WIDT
  if(value1 > THRESHOLD) begin
   DATA_R1=255;
   DATA_G1=255;
   DATA_B1=255;
  end
  else begin
   DATA_R1=0;
   DATA_G1=0;
   DATA_B1=0;
  end
  `endif
 end
end

endmodule
```

The image processing operation is selected in the following "parameter.v" file. To change the processing operation, just switch the comment line.

```
/*************************************/
 /****************** Definition file ********/
/************** ***********************************************/
`define INPUTFILENAME "your_image.hex" // Input file name
`define OUTPUTFILENAME "output.bmp" // Output file name
// Choose the operation of code by delete
// in the beginning of the selected line
//`define BRIGHTNESS_OPERATION
 `define INVERT_OPERATION
//`define THRESHOLD_OPERATION
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
```

The "parameter.v" file is also to define paths and names of the input and output file. After processing the image, it is needed to write the processed data to an output image for verifications.

The following Verilog code is to write the processed image data to a bitmap image for verification:

```
/****************** Module for writing .bmp image *************/
/*************************************************************/
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Image processing in Verilog
module image_write #(parameter
WIDTH = 768, // Image width
HEIGHT = 512, // Image height
INFILE = "output.bmp", // Output image
BMP_HEADER_NUM = 54 // Header for bmp image
)
(
input HCLK, // Clock input
HRESETn, // Reset active low
input hsync, // Hsync pulse
input [7:0] DATA_WRITE_R0, // Red 8-bit data (odd)
input [7:0] DATA_WRITE_G0, // Green 8-bit data (odd)
input [7:0] DATA_WRITE_B0, // Blue 8-bit data (odd)
input [7:0] DATA_WRITE_R1, // Red 8-bit data (even)
input [7:0] DATA_WRITE_G1, // Green 8-bit data (even)
input [7:0] DATA_WRITE_B1, // Blue 8-bit data (even)
output reg Write_Done
);
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
//-----------------------------------//
//-------Header data for bmp image-----//
//-------------------------------------//
// Windows BMP files begin with a 54-byte header
initial  begin
BMP_header[ 0] = 66;BMP_header[28] =24;
BMP_header[ 1] = 77;BMP_header[29] = 0;
BMP_header[ 2] = 54;BMP_header[30] = 0;
BMP_header[ 3] = 0;BMP_header[31] = 0;
BMP_header[ 4] = 18;BMP_header[32] = 0;
BMP_header[ 5] = 0;BMP_header[33] = 0;
BMP_header[ 6] = 0;BMP_header[34] = 0;
BMP_header[ 7] = 0;BMP_header[35] = 0;
BMP_header[ 8] = 0;BMP_header[36] = 0;
BMP_header[ 9] = 0;BMP_header[37] = 0;
BMP_header[10] = 54;BMP_header[38] = 0;
BMP_header[11] = 0;BMP_header[39] = 0;
BMP_header[12] = 0;BMP_header[40] = 0;
BMP_header[13] = 0;BMP_header[41] = 0;
BMP_header[14] = 40;BMP_header[42] = 0;
BMP_header[15] = 0;BMP_header[43] = 0;
BMP_header[16] = 0;BMP_header[44] = 0;
BMP_header[17] = 0;BMP_header[45] = 0;
BMP_header[18] = 0;BMP_header[46] = 0;
BMP_header[19] = 3;BMP_header[47] = 0;
BMP_header[20] = 0;BMP_header[48] = 0;
BMP_header[21] = 0;BMP_header[49] = 0;
BMP_header[22] = 0;BMP_header[50] = 0;
BMP_header[23] = 2;BMP_header[51] = 0;
BMP_header[24] = 0;BMP_header[52] = 0;
BMP_header[25] = 0;BMP_header[53] = 0;
BMP_header[26] = 1; BMP_header[27] = 0;
end
//-------------------------------------------------------//
//-------------Write .bmp file  ----------------------//
//-------------------------------------------------------//
initial begin
    fd = $fopen(INFILE, "wb+");
end
always@(Write_Done) begin // once the processing was done, bmp image will be creat
```

```
    if(Write_Done == 1'b1) begin
        for(i=0; i<BMP_HEADER_NUM; i=i+1) begin
            $fwrite(fd, "%c", BMP_header[i][7:0]); // write the header
        end

        for(i=0; i<WIDTH*HEIGHT*3; i=i+6) begin
  // write R0B0G0 and R1B1G1 (6 bytes) in a loop
            $fwrite(fd, "%c", out_BMP[i  ][7:0]);
            $fwrite(fd, "%c", out_BMP[i+1][7:0]);
            $fwrite(fd, "%c", out_BMP[i+2][7:0]);
            $fwrite(fd, "%c", out_BMP[i+3][7:0]);
            $fwrite(fd, "%c", out_BMP[i+4][7:0]);
            $fwrite(fd, "%c", out_BMP[i+5][7:0]);
        end
    end
end
```

The header data for the bitmap image is very important and it is published here. If there is no header data, the written image could not be correctly displayed. In Verilog HDL, $fwrite command is used to write data to file.

Next, let's write a test bench Verilog code to verify the image processing operations.

```
`timescale 1ns/1ps /**************************************************/
/******* Testbench for simulation ****************/
/**********************************************/
 // fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Image processing in Verilog
`include "parameter.v" // include definition file module tb_simulation;
//----------------- // Internal Signals
//-------------------------------------------------
reg HCLK, HRESETn;
wire vsync;
wire hsync;
wire [ 7 : 0] data_R0;
wire [ 7 : 0] data_G0;
wire [ 7 : 0] data_B0;
wire [ 7 : 0] data_R1;
wire [ 7 : 0] data_G1;
wire [ 7 : 0] data_B1;
wire enc_done;
image_read #(.INFILE(`INPUTFILENAME))
u_image_read
( .HCLK (HCLK ),
.HRESETn (HRESETn ),
 .VSYNC (vsync ),
.HSYNC (hsync ),
.DATA_R0 (data_R0 ),
 .DATA_G0 (data_G0 ),
.DATA_B0 (data_B0 ),
.DATA_R1 (data_R1 ),
.DATA_G1 (data_G1 ),
.DATA_B1 (data_B1 ),
.ctrl_done (enc_done)
);
image_write #(.INFILE(`OUTPUTFILENAME))
u_image_write (
.HCLK(HCLK),
.HRESETn(HRESETn),
 .hsync(hsync),
.DATA_WRITE_R0(data_R0),
 .DATA_WRITE_G0(data_G0),
 .DATA_WRITE_B0(data_B0),
.DATA_WRITE_R1(data_R1),
.DATA_WRITE_G1(data_G1),
```

```
.DATA_WRITE_B1(data_B1),
 .Write_Done()
 );
//------------- // Test Vectors
//-------------------------------------
initial
begin
HCLK = 0;
forever #10 HCLK = ~HCLK;
end
initial
begin
HRESETn = 0;
#25 HRESETn = 1;
end endmodule
```

Finally, we have everything to run a simulation to verify the image processing code. Let's use the following image as the input bitmap file:

Input bitmap image

Run the simulation for 6ms, close the simulation and open the output image for checking the result. Followings are the output images which are processed by the selected operations in parameter.v:

Output bitmap image after inverting

Output bitmap image after threshold operation

Output bitmap image after subtracting brightness

Since the reading code is to model an image sensor/camera for simulation purposes, it is recommended not to synthesize the code. If you really want to synthesize the processing code and run this directly on FPGA, you need to replace these image arrays (total_memory, temp_BMP, org_R, org_B, org_G) in the code by block memory (RAMs) and design address generators to read image data from the block memory.

After receiving so many questions related to this project, followings are the answers to your questions:

1. Explanation and tutorial on how to run the simulation:

2. The full Verilog code for this image processing project can be downloaded here. Run the simulation about 6ms and close the simulation, then you will be able to see the output image.

3. The reading part operates as a Verilog model of an image sensor/camera (output RGB data, HSYNC, VSYNC, HCLK). The Verilog image reading code is extremely useful for functional verification in real-time FPGA image/video projects.

4. In this project, I added the image processing part to make an example of image enhancement. You can easily remove the processing part to get only raw image data in the case that you want to use the image sensor model only for verifying your image processing design.

5. The image saving into three separate RGB mems: Because Matlab code writes the image hexadecimal file from the last row to the first row, the RGB saving codes (org_R, org_B, org_G) do the same in reading the temp_BMP memory to save RGB data correctly. You can change it accordingly if you want to do it differently.

6. You might find the following explanation for the BMP header useful if you want to change the image size:

*Image size = 768*512*3= 1179648 bytes*
BMP header = 54 bytes
BMP File size = Image size + BMP Header = 1179702 Bytes
Convert it to hexadecimal numbers: 1179702 in Decimal = 120036 in Hexadecimal
Then 4-byte size of BMP file: 00H, 12 in Hexa = 18 Decimal, 00H, 36 in Hexa = 54 Decimal
That's how we get the following values:
BMP_header[2] = 54;
BMP_header[3] = 0 ;
BMP_header[4] = 18;
BMP_header[5] = 0 ;
Image width = 768 => In hexadecimal: 0x0300. The 4 bytes of the image width are 0, 3, 0, 0. That's how you get the following values:

BMP_header[18] = 0;
BMP_header[19] = 3;
BMP_header[20] = 0;
BMP_header[21] = 0;
Image height = 512 => In hexadecimal: 0x0200. The 4 bytes of the image width are 0, 2, 0, 0. That's how we get the following values:
BMP_header[22] = 0;
BMP_header[23] = 2;
BMP_header[24] = 0;
BMP_header[25] = 0;

7. You should not synthesize this code because it is not designed for running on FPGA, *but rather for functional verification purposes*. If you really want to synthesize this code (read and process) and load the image into FPGA for processing directly on FPGA, replace all the temp. variables (org_R, org_B, org_G, tmp_BMP = total_memory) by block RAMs and generate addresses to read the image data (remove the always @(start) and all the "for loops" - these are for simulation purposes). There are two ways: 1. write a RAM code and initialize the image data into the memory using $readmemh; 2. generate a block memory using either Xilinx Core Generator or Altera MegaFunction and load the image data into the initial values of memory (.coe file for Xilinx Core Gen. and .mif for Altera MegaFunction), then read the image data from memory and process it (FSM Design).

8. In this project, two even and old pixels are read at the same time for speeding up the processing, but you can change the number of pixels being read depending on your design.

9. The writing Verilog code is also very helpful for testing purposes as you can see the output in BMP format.

10. If you want to do real-time image processing, you can check this for the camera interface code: **Basys 3 FPGA OV7670 Camera**

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. **Verilog code for FIFO memory**
3. **Verilog code for 16-bit single-cycle MIPS processor**
4. **Programmable Digital Delay Timer in Verilog HDL**
5. **Verilog code for basic logic components in digital circuits**
6. **Verilog code for 32-bit Unsigned Divider**
7. **Verilog code for Fixed-Point Matrix Multiplication**
8. **Plate License Recognition in Verilog HDL**
9. **Verilog code for Carry-Look-Ahead Multiplier**
10. **Verilog code for a Microcontroller**
11. **Verilog code for 4x4 Multiplier**
12. **Verilog code for Car Parking System**
13. **Image processing on FPGA using Verilog HDL**
14. **How to load a text file into FPGA using Verilog HDL**
15. **Verilog code for Traffic Light Controller**
16. **Verilog code for Alarm Clock on FPGA**
17. **Verilog code for comparator design**
18. **Verilog code for D Flip Flop**
19. **Verilog code for Full Adder**
20. **Verilog code for counter with testbench**
21. **Verilog code for 16-bit RISC Processor**
22. **Verilog code for button debouncing on FPGA**
23. **How to write Verilog Testbench for bidirectional/ inout ports**
24. **Tic Tac Toe Game in Verilog and LogiSim**
25. **32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)**
26. **32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)**
27. **32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)**
28. **Verilog code for Decoder**
29. **Verilog code for Multiplexers**
30. **N-bit Adder Design in Verilog**
31. **Verilog vs VHDL: Explain by Examples**

32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Moore FSM Sequence Detector**
37. **Verilog code for 7-segment display controller on Basys 3 FPGA**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.